Raspberry Pi 5
Funcionamiento profundo

Hardware, software y subsistemas
para el payload CubeSat-EdgeAI

Plataforma:	Raspberry Pi 5 (4 GB) — desarrollo
Modelo de vuelo:	Compute Module 5 (proximamente)
Proyecto:	CubeSat-EdgeAI-Payload
Repositorio:	`github.com/JaminYC/CubeSat-EdgeAI-Payload`
Fecha:	5 de mayo de 2026
Documento:	v1.0

Resumen. Documento de referencia integral del Raspberry Pi 5 enfocado en su uso como payload de procesamiento autonomo a bordo de un CubeSat. Cubre el SoC BCM2712, la arquitectura ARMv8, el sistema de memoria LPDDR4X, los buses GPIO (I2C, SPI, UART), el subsistema de camara CSI-2, la PMIC RP1, el proceso de boot, el sistema operativo Raspberry Pi OS, la gestion termica, y la migracion al Compute Module 5 para el modelo de vuelo. Cada capitulo incluye notas especificas para integracion al INTISAT, comandos practicos de Linux y advertencias de hardware.

Índice general

1. Vision general **2**
1.1. Arquitectura de bloques . . . 2
1.2. Diferencia clave vs Pi 4 . . . 2

2. El SoC BCM2712 **3**
2.1. CPU — ARM Cortex-A76 . . . 3
2.2. GPU — VideoCore VII . . . 3
2.3. ISP (Image Signal Processor) . . . 3
2.4. Memoria LPDDR4X . . . 4

3. El controlador RP1 (clave para entender el Pi 5) **5**
3.1. Que es . . . 5
3.2. Que controla . . . 5
3.3. Por que importa para vos . . . 5

4. Sistema de potencia **6**
4.1. Rails de tension . . . 6
4.2. Consumo (medido) . . . 6
4.3. Brown-out detection . . . 6

5. El header de 40 pines (GPIO) **7**
5.1. Pinout completo . . . 7
5.2. Pines usados por tu payload . . . 7
5.3. Funciones alternativas (alt modes) . . . 7

6. Buses de comunicacion **8**
6.1. I2C . . . 8
6.1.1. Direcciones . . . 8
6.1.2. Modo slave (BSC) . . . 8
6.1.3. Test de comandos basicos . . . 8
6.2. SPI . . . 8
6.3. UART . . . 9
6.4. CSI-2 (camara) . . . 9
6.5. PCIe . . . 9

7. Almacenamiento **10**
7.1. Tarjeta micro-SD . . . 10
7.2. NVMe (via PCIe HAT) . . . 10
7.3. eMMC (solo CM5) . . . 10

8. Proceso de boot **11**
8.1. Etapas del boot . . . 11
8.2. config.txt importante para tu payload . . . 11
8.3. Boot order . . . 11
8.4. Recovery / fastboot . . . 11

9. Sistema operativo **12**
9.1. Raspberry Pi OS . . . 12
9.2. Kernel . . . 12
9.3. Systemd . . . 12
9.4. Servicios del payload . . . 12

10. Programacion de GPIO en Pi 5 **13**
10.1. Librerias disponibles . . . 13
10.2. Ejemplo: blink de LED en GPIO17 . . . 13
10.3. Ejemplo: leer sensor con I2C . . . 13

11. Camara y video **14**
11.1. libcamera + Picamera2 . . . 14
11.2. Configuracion para microscopia cuantitativa . . . 14
11.3. Sensor OV5647 specs . . . 14

12. Gestion termica **15**
12.1. Sensores . . . 15
12.2. Throttling . . . 15
12.3. Disipacion en CubeSat . . . 15

13. Networking (en lab, no en orbita) **16**
13.1. Ethernet Gigabit . . . 16
13.2. WiFi y Bluetooth . . . 16
13.3. En el modelo de vuelo . . . 16

14. Migracion al Compute Module 5 **17**
14.1. Diferencias clave . . . 17
14.2. Carrier board para CubeSat . . . 17
14.3. Software . . . 17

15. Robustez para vuelo **18**
15.1. Watchdog hardware . . . 18
15.2. Memoria SD vs eMMC en orbita . . . 18
15.3. Bit rot y radiacion . . . 18
15.4. Uptime y reboots planificados . . . 18

16. Comandos utiles de diagnostico **19**
16.1. Hardware . . . 19
16.2. Buses . . . 19
16.3. Camara . . . 19
16.4. Servicios . . . 19

17. Diagnosticos comunes y soluciones **21**
17.1. El sistema no botea . . . 21
17.2. I2C no detecta dispositivos . . . 21
17.3. Camara no detectada . . . 21
17.4. Temperatura alta . . . 21

18. Checklist final pre-vuelo **22**

Referencias oficiales **23**

1. Vision general

El Raspberry Pi 5 es una computadora single-board de credit-card size que combina un SoC moderno (BCM2712), 4–16 GB de RAM, conectividad multiproposito y un IO controller dedicado (RP1). Para tu payload actua como *computadora principal del experimento*: corre Linux, controla los sensores opticos por buses estandar, ejecuta el pipeline de IA y dialoga con el OBC del satelite por I2C.

1.1. Arquitectura de bloques

SoC BCM2712	4 nucleos ARM Cortex-A76 @ 2.4 GHz, GPU VideoCore VII, ISP, decoder H.265
RP1 IO controller	chip dedicado (Raspberry Pi diseñado) que expone GPIO, I2C, SPI, UART, USB, Ethernet
LPDDR4X RAM	4/8/16 GB, bus 64 bit, 4267 MT/s
PCIe Gen 2 x1	FPC connector externo, hasta 5 Gbit/s — para NVMe SSD
2x USB 3.0 + 2x USB 2.0	via RP1 |
2x camera CSI-2 + 2x display DSI	pueden funcionar en simultaneo
40-pin GPIO header	compatible con Pi anterior pero con drivers nuevos en RP1

1.2. Diferencia clave vs Pi 4

El cambio mas importante: **el GPIO ya no esta directamente en el SoC**. Pasa por el RP1, que es un chip independiente con su propio firmware. Esto trae implicaciones:

- Latencia GPIO mas alta ($\sim 1\ \mu$s vs ns en Pi 4).
- `RPi.GPIO` y `wiringpi` no funcionan; hay que usar `lgpio` o `gpiozero`.
- `pigpio` todavia funciona (necesario para tu I2C esclavo).
- Frecuencias maximas: I2C hasta 1 MHz, SPI hasta 50 MHz.

Para tu payload

El `cubesat/i2c_slave.py` usa `pigpio.bsc_i2c()` que funciona en Pi 5 con el demonio pigpiod corriendo. No usa el bus I2C clasico (que solo soporta master), sino el BSC (Broadcom Serial Controller) que si soporta modo slave.

2. El SoC BCM2712

2.1. CPU — ARM Cortex-A76

Cuatro nucleos identicos, cada uno con:

- Pipeline out-of-order de 12 etapas, 4 instrucciones decodificadas por ciclo.
- Cache L1 de instrucciones 64 KB + L1 de datos 64 KB.
- Cache L2 privada por nucleo de 512 KB.
- Cache L3 compartida de 2 MB.
- Instrucciones SIMD: NEON (Advanced SIMD) y SVE (Scalable Vector Extension).
- Frecuencia base 2.4 GHz, dynamic frequency scaling de 1.5 a 2.4 GHz.

Performance practico (single-core, comparativa con Pi 4):

Benchmark	**Pi 4**	**Pi 5**
Sysbench CPU prime	1.5x ref	4.2x ref
Geekbench 6 single	220	770
Geekbench 6 multi	600	1700

Aproximadamente 3× mas rapido que Pi 4 en cargas single-thread.

2.2. GPU — VideoCore VII

- 800 MHz, soporta OpenGL ES 3.1 y Vulkan 1.2.
- Decoder H.265 hardware (4kp60), decoder H.264 (1080p60), encoder H.264 (1080p30).
- NO soporta CUDA (no es NVIDIA).

Para tu payload: la GPU NO se usa para IA (no hay framework optimizado ARM Mali/VideoCore para PyTorch). El pipeline IA corre en CPU.

2.3. ISP (Image Signal Processor)

Procesa la salida cruda de la camara CSI-2 antes de pasarla a memoria:

- Bayer demosaicing.
- Auto white balance (AWB).
- Auto exposure (AE).
- Lens shading correction.
- Noise reduction (NR).

Cuidado

El ISP *aplica correcciones automaticas* que pueden interferir con microscopia cuantitativa. En `cubesat/capture.py` se desactivan con `Picamera2.set_controls({"AwbEnable": False, "AeEnable": False})`.

2.4. Memoria LPDDR4X

- 4267 MT/s (efectivo $\sim$ 17 GB/s en burst).
- Bus 64 bit (4 canales x16 internos).
- Latencia $\sim$ 80 ns (timing CL40).
- Versiones: $4, 8, 16$ GB. La de tu payload es 4 GB.

3. El controlador RP1 (clave para entender el Pi 5)

3.1. Que es

El RP1 es un chip diseñado por Raspberry Pi Ltd (no Broadcom) que actua como *southbridge*: maneja todos los IO perifericos. Esta interconectado al SoC por PCIe Gen 2 x4.

3.2. Que controla

- 40-pin GPIO header (todos los pines).
- 2 buses I2C (I2C1 y dedicated).
- 5 buses SPI.
- 6 UARTs.
- Ethernet Gigabit.
- 2x USB 3.0 + 2x USB 2.0.
- Audio analog out, PWM, etc.

3.3. Por que importa para vos

a) **Latencia**: el viaje SoC $\rightarrow$ PCIe $\rightarrow$ RP1 $\rightarrow$ pin agrega ~ 1 μs. No es problema para microscopia, pero seria limitante para señales en tiempo real > 100 kHz.

b) **Drivers nuevos**: si una libreria asume que el GPIO esta en el SoC (como `RPi.GPIO`), no funciona. Usar `lgpio`, `gpiozero` o `pigpio`.

c) **I2C slave**: solo el BSC original del SoC soporta modo slave; el RP1 expone solo modo master. El `cubesat/i2c_slave.py` usa el BSC directamente con `pigpio.bsc_i2c()`.

4. Sistema de potencia

4.1. Rails de tension

El Pi 5 entra con 5 V (USB-C o pin 2/4 del header) y la PMIC interna genera:

Rail	Tension	Para que
V_{IN}	5.0 V	entrada externa
V_{CORE}	0.85–0.9 V	nucleos CPU (variable, DVFS)
V_{GPU}	0.9 V	GPU
V_{DDR}	1.1 V	RAM LPDDR4X
$V_{\mathrm{IO_18}}$	1.8 V	buses internos del SoC
$V_{\mathrm{IO_33}}$	3.3 V	GPIO + perifericos externos

4.2. Consumo (medido)

Estado	Corriente @ 5V (A)	Potencia (W)
Boot peak	1.6 A	8.0
Idle (HDMI desconectado)	0.6 A	3.0
CPU 100 % (1 core)	0.9 A	4.5
CPU 100 % (4 cores)	1.4 A	7.0
CPU + GPU + WiFi + USB	1.6 A	8.0

Mediciones tipicas; la POE+ HAT oficial provee hasta 5 A.

4.3. Brown-out detection

Si V_{IN} cae bajo 4.6 V el Pi entra en *undervoltage*, se ve un icono de rayo en pantalla y la CPU baja a 1.5 GHz. En log de kernel:

```
[123.456] Under-voltage detected! (0x00050005)
```

Verificar con `vcgencmd get_throttled`: el bit 0 indica undervoltage.

Cuidado

En la integracion al INTISAT, el rail 5V_PAYLOAD debe sostener 1.6 A de pico sin caer abajo de 4.7 V. El fusible PTC de 5 A elegido tiene margen, pero verificar regulacion del LDO bajo carga.

5. El header de 40 pines (GPIO)

5.1. Pinout completo

Pin	Funcion	BCM	Tension	Pin	Funcion	BCM	Tension
1	3.3V	—	3.3V	2	5V	—	5V
3	SDA1 / I2C	GPIO2	3.3V	4	5V	—	5V
5	SCL1 / I2C	GPIO3	3.3V	6	GND	—	0V
7	GPIO4	GPIO4	3.3V	8	TXD0 / UART	GPIO14	3.3V
9	GND	—	0V	10	RXD0 / UART	GPIO15	3.3V
11	GPIO17	GPIO17	3.3V	12	GPIO18	GPIO18	3.3V
13	GPIO27	GPIO27	3.3V	14	GND	—	0V
15	GPIO22	GPIO22	3.3V	16	GPIO23	GPIO23	3.3V
17	3.3V	—	3.3V	18	GPIO24	GPIO24	3.3V
19	MOSI / SPI	GPIO10	3.3V	20	GND	—	0V
21	MISO / SPI	GPIO9	3.3V	22	GPIO25	GPIO25	3.3V
23	SCLK / SPI	GPIO11	3.3V	24	CE0 / SPI	GPIO8	3.3V
25	GND	—	0V	26	CE1 / SPI	GPIO7	3.3V
27	ID_SDA	GPIO0	3.3V	28	ID_SCL	GPIO1	3.3V
29	GPIO5	GPIO5	3.3V	30	GND	—	0V
31	GPIO6	GPIO6	3.3V	32	GPIO12	GPIO12	3.3V
33	GPIO13	GPIO13	3.3V	34	GND	—	0V
35	GPIO19	GPIO19	3.3V	36	GPIO16	GPIO16	3.3V
37	GPIO26	GPIO26	3.3V	38	GPIO20	GPIO20	3.3V
39	GND	—	0V	40	GPIO21	GPIO21	3.3V

5.2. Pines usados por tu payload

Pin	Funcion	Conectado a
2 / 4	5V IN	EPS via fusible + LDO
6, 14, 20, 30, 39	GND	GND comun
3 (GPIO2)	SDA I2C_2	TXS0108E → OBC INTISAT
5 (GPIO3)	SCL I2C_2	TXS0108E → OBC INTISAT
12 (GPIO18)	OLED DC	SSD1351
18 (GPIO24)	OLED RST	SSD1351
19 (GPIO10)	MOSI SPI0	SSD1351
23 (GPIO11)	SCLK SPI0	SSD1351
24 (GPIO8)	CE0 SPI0	SSD1351

5.3. Funciones alternativas (alt modes)

Cada GPIO puede configurarse en hasta 6 funciones distintas. Por ejemplo GPIO12 puede ser GPIO digital, PWM, SPI3 MOSI, UART4 TX, etc. Se selecciona con `raspi-gpio set <pin> a<n>` o desde `config.txt`.

```
# Ver configuracion actual
raspi-gpio get 12
# Cambiar a PWM
raspi-gpio set 12 a0
```

6. Buses de comunicacion

6.1. I2C

Bus de 2 hilos (SDA, SCL) con multimaster. Velocidades estandar 100 kHz, 400 kHz, 1 MHz, 3.4 MHz (high-speed mode no soportado).

6.1.1. Direcciones

I2C usa direcciones de 7 bit. El payload ocupa **0x42**. Direcciones reservadas:

- `0x00`: general call.
- `0x08-0x0F`: high-speed master.
- `0x78-0x7F`: 10-bit addressing.

6.1.2. Modo slave (BSC)

El SoC tiene un periferico llamado BSC (Broadcom Serial Controller) que puede actuar como slave. Solo accesible via `pigpio.bsc_i2c()`, no por la libreria estandar.

```
import pigpio
pi = pigpio.pi()
pi.bsc_i2c(0x42)   # responde como slave en 0x42

while True:
    s, b, d = pi.bsc_i2c(0x42)   # lee bytes recibidos
    if b > 0:
        process(d[:b])
```

6.1.3. Test de comandos basicos

```
# Detectar dispositivos en el bus
sudo i2cdetect -y 1

# Leer 1 byte de address 0x42, registro 0
i2cget -y 1 0x42 0x00

# Escribir 0x05 al registro 0x10 del slave 0x42
i2cset -y 1 0x42 0x10 0x05
```

6.2. SPI

Bus sincrono full-duplex. El Pi 5 tiene 5 buses SPI (SPI0-SPI5) via RP1, mas SPI6 directo del SoC.

- Velocidades: 0.5 MHz a 50 MHz.
- Modos: 0, 1, 2, 3 (combinaciones de CPOL y CPHA).
- En tu payload: SPI0 con OLED SSD1351 a 1 MHz.

Habilitar via `raspi-config` o en `/boot/firmware/config.txt`:

```
dtparam=spi=on
```

6.3. UART

Hasta 6 UARTs disponibles (UART0-UART5). El UART0 esta conectado por defecto a la consola serial (115200, 8N1). Util para debug:

```
# Conectarse desde otra Pi por UART (pin 8 TX, pin 10 RX)
sudo screen /dev/ttyAMA0 115200
```

6.4. CSI-2 (camara)

MIPI Camera Serial Interface 2. El Pi 5 tiene 2 conectores FPC de 22 pines (diferente del Pi 4 que era de 15 pines, requiere cable adaptador).

- 4 lanes de datos diferenciales.
- Hasta 1.5 Gbps por lane.
- Soporta hasta 4032×3040 @ 30 fps (12 MP).

Para OV5647: 2 lanes, 2592×1944 @ 15 fps en 5 MP. Usa `Picamera2` library:

```
from picamera2 import Picamera2
cam = Picamera2()
config = cam.create_still_configuration(main={"size": (2592, 1944)})
cam.configure(config)
cam.start()
img = cam.capture_array()
```

6.5. PCIe

PCIe Gen 2 x1 expuesto en el conector FPC lateral. Velocidad 5 Gbps. Los HATs oficiales permiten conectar SSD NVMe.

Para tu payload

Para el modelo de vuelo CM5, considerar agregar SSD NVMe en lugar de SD para almacenar resultados. La SD se corrompe facilmente con vibracion y radiacion. NVMe + eMMC (ya integrada en el CM5) es mucho mas robusta.

7. Almacenamiento

7.1. Tarjeta micro-SD

Slot estandar SDIO 3.0, soporta hasta UHS-I (104 MB/s). Es el medio principal en Pi 5 retail. **Formato recomendado**:

- Tabla de particiones MBR (no GPT).
- Particion 1: FAT32 `boot` ($\geq$ 256 MB).
- Particion 2: ext4 `rootfs` (resto).

Cuidado

Las SD baratas se degradan rapido con escrituras. Para el payload de vuelo NO USAR SD. Migrar a CM5 con eMMC integrada.

7.2. NVMe (via PCIe HAT)

Velocidades reales $\sim$ 800 MB/s read, 700 MB/s write con un Samsung 980. Booteo desde NVMe disponible en EEPROM $\geq 2024 - 09$.

7.3. eMMC (solo CM5)

El Compute Module 5 trae eMMC integrada de 0 (Lite), 4, 16, 32, 64 GB. Mas robusta que SD frente a vibracion y temperatura. Velocidad $\sim$ 200 MB/s.

8. Proceso de boot

8.1. Etapas del boot

1. **Bootloader EEPROM** (interno al SoC): inicializa el SoC, habilita LPDDR4X, busca medio de boot segun `BOOT_ORDER`.
2. **start.elf** (en partition de boot): carga el firmware del GPU.
3. **config.txt**: archivo de configuracion (resoluciones, overclocks, device tree overlays).
4. **cmdline.txt**: argumentos del kernel.
5. **Kernel Linux** (`kernel_2712.img`): carga el sistema.
6. **initramfs**: filesystem temporal con drivers.
7. **systemd**: lanza servicios de userspace.
8. **Login**: sistema listo.

8.2. config.txt importante para tu payload

```
# Habilitar buses
dtparam=i2c_arm=on
dtparam=spi=on
dtparam=audio=off                    # OLED SSD1351 reusa pines audio

# Camara CSI-2
camera_auto_detect=1
dtoverlay=ov5647

# Watchdog hardware (cuelgue $\to$ reboot automatico)
dtparam=watchdog=on

# I2C slave en GPIO2/3
dtoverlay=i2c-gpio,bus=10,i2c_gpio_sda=2,i2c_gpio_scl=3
```

8.3. Boot order

Configurable en EEPROM. Por defecto:

```
BOOT_ORDER=0xf41   # SD, NVMe, USB, repeat
```

Editar con `sudo rpi-eeprom-config -e`.

8.4. Recovery / fastboot

Si el sistema no bootea, mantener pulsado el boton del Pi 5 al darle power para entrar a un mini OS de recovery. Solo en Pi 5, no Pi 4.

9. Sistema operativo

9.1. Raspberry Pi OS

Distribucion oficial basada en Debian. Version actual al momento de redactar: **Bookworm** (Debian 12). Imagenes:

- **Lite**: sin escritorio, ideal para servidor / payload (lo que usamos).
- **Desktop**: con LXDE, 5 GB.
- **Full**: con LibreOffice, Chromium, etc., 10 GB.

Bajar de https://www.raspberrypi.com/software/operating-systems/.

9.2. Kernel

Linux $\sim$ 6.6 con patches RPi. Verificar:

```
uname -a
# Linux raspberrypi 6.6.20+rpt-rpi-2712 #1 SMP PREEMPT Debian
```

PREEMPT indica preemptive scheduling (mejor latencia).

9.3. Systemd

Sistema de init y manager de servicios. Comandos basicos:

```
# Listar servicios activos
systemctl list-units --type=service --state=running

# Habilitar / deshabilitar
sudo systemctl enable pigpiod
sudo systemctl start pigpiod

# Ver estado
systemctl status cubesat-pipeline

# Logs
journalctl -u cubesat-pipeline -f --since "1 hour ago"
```

9.4. Servicios del payload

- `pigpiod.service`: demonio pigpio (necesario para I2C slave).
- `cubesat-pipeline.service`: el daemon principal con WatchdogSec.
- `cubesat-i2c-slave.service`: dispatcher I2C en 0x42.
- `cubesat-telemetry.service`: snapshots de telemetria cada 30 s.

10. Programacion de GPIO en Pi 5

10.1. Librerias disponibles

Libreria	Pi 4	Pi 5
`RPi.GPIO`	si	**NO** (deprecada)
`wiringpi`	si	**NO** (no portada)
`lgpio`	si	**si** (recomendada)
`gpiozero`	si	**si** (alto nivel)
`pigpio`	si	**si** (con demonio)
`libgpiod`	si	**si** (la oficial Linux)

10.2. Ejemplo: blink de LED en GPIO17

```
import gpiozero
import time
led = gpiozero.LED(17)
while True:
    led.on()
    time.sleep(0.5)
    led.off()
    time.sleep(0.5)
```

10.3. Ejemplo: leer sensor con I2C

```
import smbus2
bus = smbus2.SMBus(1)          # I2C-1
addr = 0x68                    # MPU-6050 IMU
data = bus.read_byte_data(addr, 0x75)   # WHO_AM_I
print(hex(data))               # debe imprimir 0x68
```

11. Camara y video

11.1. libcamera + Picamera2

`libcamera` es el stack moderno de camaras de Linux, reemplaza al viejo `raspistill`. `Picamera2` es el wrapper Python.

11.2. Configuracion para microscopia cuantitativa

```
from picamera2 import Picamera2
import numpy as np

cam = Picamera2()
config = cam.create_still_configuration(
    main={"size": (2592, 1944), "format": "RGB888"},
    raw={"size": (2592, 1944)},
)
cam.configure(config)

# Desactivar correcciones automaticas
cam.set_controls({
    "AwbEnable": False,
    "AeEnable": False,
    "ExposureTime": 50000,    # 50 ms
    "AnalogueGain": 1.0,
    "ColourGains": (1.0, 1.0),
})
cam.start()

# Capturar
img = cam.capture_array("main")     # numpy array (H, W, 3)
raw = cam.capture_array("raw")      # Bayer 10-bit
```

11.3. Sensor OV5647 specs

- Resolucion activa: 2592×1944 (5 MP).
- Pixel pitch: 1.4 μm.
- Profundidad: 10 bits crudos (RAW10).
- Frame rate: 30 fps @ 1080p, 15 fps @ 5 MP.
- Lente original: removida para uso lensless.

12. Gestion termica

12.1. Sensores

El SoC BCM2712 tiene un sensor de temperatura interno accesible por:

```
vcgencmd measure_temp
# temp=45.2'C

cat /sys/class/thermal/thermal_zone0/temp
# 45200 (mili-grados Celsius)
```

12.2. Throttling

Cuando $T > 80°C$ el SoC reduce frecuencia automaticamente:

- $T > 80°C$: throttle suave (-200 MHz).
- $T > 85°C$: throttle agresivo (1.5 GHz).
- $T > 90°C$: posible apagado por proteccion.

Verificar con:

```
vcgencmd get_throttled
# throttled=0x0     (sin throttling)
# throttled=0x40000  (en throttling actual)
```

12.3. Disipacion en CubeSat

Cuidado

El CubeSat NO tiene aire para conveccion. La disipacion del Pi 5 (hasta 7 W en CPU 100 %) genera ΔT alto. Soluciones:

- Disipador termico de aluminio en contacto directo con el SoC.
- Pad termico o pasta hacia la estructura del satelite (que actua como radiador).
- Limitar el duty cycle del modo PROCESSING.
- Considerar undervoltaje y limite de frecuencia:
  `arm_freq=1500` en `config.txt` reduce a 1.5 GHz.

13. Networking (en lab, no en orbita)

13.1. Ethernet Gigabit

PHY integrada en RP1, conector RJ-45. Soporta PoE+ (con HAT oficial).

13.2. WiFi y Bluetooth

Cypress CYW43455 dual-band (2.4 + 5 GHz) WiFi 802.11ac + Bluetooth 5.0 BLE. En CubeSat no se usa (pero util para debug en lab):

```
# Configurar WiFi sin GUI
sudo nmtui
```

13.3. En el modelo de vuelo

CM5 **NO incluye** WiFi/BT por defecto (hay version con). Para CubeSat se desactivan via `config.txt`:

```
dtoverlay=disable-wifi
dtoverlay=disable-bt
```

14. Migracion al Compute Module 5

14.1. Diferencias clave

Aspecto	Pi 5	CM5
Tamaño	85 × 56 mm	55 × 40 mm
Conector	headers (40-pin etc.)	mezzanine 200-pin
Carrier	propia placa	se diseña aparte
USB	4 puertos	no expuestos
Ethernet	RJ-45	solo PHY (necesita conector externo)
HDMI	si	exposed via mezzanine
eMMC	no	**si (4–64 GB)**
Consumo	3–8 W	2.5–7 W (∼ 10 % menos)
SD	si	solo en version Lite

14.2. Carrier board para CubeSat

El carrier debe incluir:

1. Conector mezzanine de 200 pines (Hirose DF40 stack height 1.5 mm).
2. Footprint mecanico PC-104.
3. Level shifter TXS0108E para I2C externo.
4. Fusible PTC + LDO para el rail 5 V de la EPS.
5. INA219 para monitor de potencia.
6. Conector CSI-2 (15 o 22 pines) para OV5647.
7. Conector SPI + GPIO para OLED SSD1351.
8. LED de estado de boot.
9. Pads de prueba para los buses.

14.3. Software

El CM5 corre el mismo Raspberry Pi OS que el Pi 5. La transicion es transparente para el codigo: solo cambia el carrier y la imagen de boot (habilitar eMMC en lugar de SD).

Para tu payload

La migracion al CM5 NO requiere cambios en `cubesat/` ni en `pipeline/`. El unico cambio sera en `cubesat/install.sh` para flashear la eMMC en lugar de la SD.

15. Robustez para vuelo

15.1. Watchdog hardware

El BCM2712 tiene un watchdog interno que reinicia el SoC si no recibe kick. Habilitarlo:

```
# config.txt
dtparam=watchdog=on

# /etc/systemd/system/cubesat-pipeline.service
WatchdogSec=600
```

Si el daemon se cuelga ≥ 600 s sin llamar `sd_notify(WATCHDOG=1)`, systemd lo reinicia. Si systemd se cuelga, el watchdog hardware reinicia el SoC.

15.2. Memoria SD vs eMMC en orbita

Aspecto	SD	eMMC (CM5)
Vibracion	sale del slot (riesgo)	soldada
Radiacion	sin proteccion	ECC interna
Vida util	1000-3000 ciclos	10000-20000 ciclos
Velocidad	100 MB/s	200 MB/s

15.3. Bit rot y radiacion

En LEO se reciben $\sim 5 - 10$ rad/dia. El BCM2712 no es rad-hard pero:

- TID toleramos ~ 10 krad antes de degradacion notable = unos 3 años de mision.
- SEU (Single Event Upset) en RAM: bit-flip aislado, mitigable con ECC software o reboots periodicos.
- SEFI (Single Event Functional Interrupt): cuelgue del SoC $\rightarrow$ watchdog hardware reinicia.

15.4. Uptime y reboots planificados

Reboot semanal preventivo:

```
# /etc/cron.d/weekly-reboot
0 3 * * 0 root /sbin/shutdown -r now
```

16. Comandos utiles de diagnostico

16.1. Hardware

```
# Modelo y revision
cat /proc/device-tree/model
# Raspberry Pi 5 Model B Rev 1.0

# Frecuencia actual de cada core
for i in 0 1 2 3; do
    cat /sys/devices/system/cpu/cpu$i/cpufreq/scaling_cur_freq
done

# Memoria
free -m

# Temperatura
vcgencmd measure_temp

# Tensiones
for v in core sdram_c sdram_i sdram_p; do
    echo -n "$v: "
    vcgencmd measure_volts $v
done

# Tirar el dump completo
vcgencmd version
vcgencmd get_throttled
```

16.2. Buses

```
# I2C scan
sudo i2cdetect -y 1

# SPI test
sudo spi-pipe -d /dev/spidev0.0 -s 1000000

# UART loopback
echo "test" | sudo socat - /dev/ttyAMA0,b115200,raw
```

16.3. Camara

```
# Listar camaras detectadas
libcamera-hello --list-cameras

# Captura rapida
libcamera-still -o test.jpg --immediate

# Sin auto-exposure
libcamera-still -o test.jpg --immediate --shutter 50000 --gain 1.0
```

16.4. Servicios

```
# Estado de todos los cubesat-*
systemctl status cubesat-* --no-pager

# Log en vivo
journalctl -f -u cubesat-pipeline

# Reiniciar todo
sudo systemctl restart cubesat-pipeline cubesat-i2c-slave cubesat-telemetry

# Ver dependencias
systemctl list-dependencies cubesat-pipeline
```

17. Diagnosticos comunes y soluciones

17.1. El sistema no botea

- LED rojo encendido continuo: error de power. Verificar fuente ≥ 5 A.
- LED verde parpadea N veces: codigo de error documentado en https://github.com/raspberrypi/firmware/issues.
- Sin LEDs: probable fallo de boot ROM. Conectar UART (pin 8/10) para ver mensajes.

17.2. I2C no detecta dispositivos

1. `lsmod | grep i2c` – ¿estan los modulos cargados?
2. `ls /dev/i2c-*` – ¿hay nodos?
3. `sudo i2cdetect -y 1` – escanea el bus.
4. Verificar pull-ups (4.7 kΩ a 3.3 V; suelen estar en la placa de sensor).
5. Confirmar tension V_{IO} correcta del slave (3.3 V para Pi GPIO).

17.3. Camara no detectada

```
# Re-habilitar
sudo raspi-config
# Interface Options $\to$ Camera $\to$ Enable

# Verificar overlay
grep -E "camera|ov5647" /boot/firmware/config.txt

# Detectar
libcamera-hello --list-cameras
```

17.4. Temperatura alta

- Verificar que el disipador este montado y tenga pasta termica.
- En `config.txt`: `arm_freq=1500` para limitar a 1.5 GHz.
- Reducir `over_voltage=-2` (undervolt seguro).
- Verificar duty cycle del pipeline IA (no correr permanentemente).

18. Checklist final pre-vuelo

- ☐ Migrado a Compute Module 5 con eMMC.
- ☐ Carrier board diseñado y validado en banco.
- ☐ Watchdog hardware habilitado (`dtparam=watchdog=on`).
- ☐ Watchdog software habilitado (`WatchdogSec=600`).
- ☐ Auto-reboot semanal configurado en cron.
- ☐ WiFi/BT deshabilitados.
- ☐ HDMI deshabilitado.
- ☐ Audio deshabilitado.
- ☐ Frecuencia limitada a 1.5 GHz.
- ☐ Disipador con pasta termica + pad a estructura.
- ☐ OTA testado en banco (git fetch + checkout + rollback).
- ☐ Burn-in 168 h sin reboot.
- ☐ Test termico –10 a +50 °C.
- ☐ Test vibracion 3 g RMS.
- ☐ Documentacion de mision completa en repo.

Referencias oficiales

- Raspberry Pi 5 product page: https://www.raspberrypi.com/products/raspberry-pi-5/
- Pi 5 documentation: https://www.raspberrypi.com/documentation/computers/raspberry-pi-5.html
- BCM2712 datasheet: https://datasheets.raspberrypi.com/bcm2712/bcm2712-peripherals.pdf
- RP1 datasheet: https://datasheets.raspberrypi.com/rp1/rp1-peripherals.pdf
- CM5 datasheet: https://datasheets.raspberrypi.com/cm5/cm5-datasheet.pdf
- CM5 IO Board: https://datasheets.raspberrypi.com/cm5/cm5-ioboard-datasheet.pdf
- Picamera2 manual: https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf
- libcamera apps: https://www.raspberrypi.com/documentation/computers/camera_software.html
- OV5647 datasheet: https://www.ovt.com/sensors/OV5647
- SSD1351 datasheet: https://www.solomon-systech.com/product/ssd1351/
- Forum oficial: https://forums.raspberrypi.com
- GitHub Pi firmware: https://github.com/raspberrypi/firmware